

7교시: 장애 시나리오 1 - API URL, DB host, 환경 변수

Week 3 Day 1 Lesson 7

개인 면담 및 MSA 환경 점검

스스로 환경을 진단하고, 문제를 해결하며, 증거를 남기는 것이 DevOps의 기본입니다.

환경 점검 체크리스트

👤 개인 면담 (1:1)

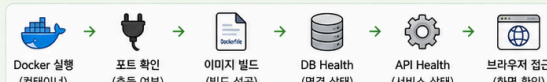
학생 체크 카드	강사 체크 카드
☐ 체크리스트 스스로 점검 완료 ☐ 문제 발생 시 원인 분석 시도 ☐ 해결 과정 기록 및 공유 ☐ 증거 스크린샷 제출 ☐ 모르는 내용 질문 정리	☐ 환경 점검 상태 확인 ☐ 문제 해결 능력 피드백 ☐ 개선 과정 안내 ☐ 다음 단계 학습 제안 ☐ 면담 내용 간단 기록
학생 서명: _____	강사 서명: _____

📷 증거 기록 (Evidence)

Figure 1 displays six panels showing Docker container logs and health checks:

- Docker 실행**: Shows the output of `$ docker ps`, listing containers `albc2344` (user-service), `bc213465` (order-service), `c34dc5fc` (mysql), and `4dc2f6g7` (rabbitmq).
- 포트 확인**: Shows the output of `$ netstat -tlnp`, confirming that port 3000 is open and listening on `0.0.0.0` by `node`.
- 이미지 빌드**: Shows the output of `$ docker build -t user-service:latest .`, indicating a successful build of the `user-service` image.
- DB Health**: Shows the output of `$ curl http://localhost:3006/health`, returning a JSON object indicating database health: `{ "status": "UP", "db": "mysql", "time": "2024-06-31" }`.
- API Health**: Shows the output of `$ curl http://localhost:3000/health`, returning a JSON object indicating API health: `{ "status": "UP", "components": { "db": { "status": "UP", "diskSpace": { "status": "up" } } } }`.
- 브라우저 접근**: Shows the output of `$ curl http://localhost:3000`, returning a 200 status code and a message: `환영합니다! MSA 개발팀입니다! 서버를 점검합니다.`

점검 순서 요약



핵심 포인트

- 스스로 문제를 발견하고 해결하는 경험이 중요합니다.
- 작은 문제라도 기록하고 공유하면 팀 전체가 성장합니다
- 증거 기반으로 설명할 수 있어야 진짜 해결한 것입니다.

! blocker 발생 시

스스로 30분 해결 시도 → 해결 안 될 경우 강사에게 도움 요청

- | | |
|----------------------|-----------------------|
| ▶ 포트 충돌 (이미 사용 중) | → 포트 변경 또는 프로세스 종료 |
| ▶ 빌드 실패 (의존성, 문법 오류) | → 로그 확인 후 수정 |
| ▶ DB 연결 실패 | → DB 컨테이너 / 환경변수 확인 |
| ▶ API Health Down | → 로그 확인 / 설정 확인 |
| ▶ 브라우저 접속 불가 | → 포트, 네트워크, API 상태 확인 |

문제 해결 과정을 기록하고, 증거를 남기는 것이 중요합니다.

기록 템플릿 (메모)

- 발생 문제: _____
- 원인 분석: _____
- 해결 방법: _____
- 결과 확인: _____
- 다음에 개선할 점: _____

수업 목표

- API 중지, DB 중지, 환경변수 오류가 서로 다른 증상을 만든다는 것을 확인한다.
- logs, inspect, exec, /health 중 무엇을 먼저 볼지 판단한다.
- 장애 리포트에 들어갈 증거를 짧고 명확하게 남긴다.

준비

```
cd week3/day1/labs/msa-demo
docker compose up --build -d
docker compose ps
```

정상 baseline을 먼저 확인한다.

```
curl -s http://localhost:18083/api/status
curl -s http://localhost:18084/health
```

장애 1: API 중지

```
docker compose stop api
curl -i http://localhost:18083/api/status
docker compose logs --tail=60 frontend
docker compose logs --tail=60 worker
```

예상 해석:

관찰	의미
frontend 요청이 502 또는 실패	nginx가 upstream api:8080에 연결하지 못함
worker log에 connection refused	worker도 API dependency에 실패
db는 healthy	DB 장애가 아니라 API service 장애 가능성

복구:

```
docker compose start api
curl -s http://localhost:18084/health
```

장애 2: DB 중지

```
docker compose stop db
curl -i http://localhost:18084/health
curl -i http://localhost:18083/api/status
docker compose logs --tail=60 api
```

예상 해석:

관찰	의미
API container는 Up	process는 살아 있음
/health가 503	readiness 실패
database_reachable=false	DB dependency 실패
frontend도 API 결과를 정상으로 표시하지 못함	사용자 경로에도 전파

복구:

```
docker compose start db
sleep 5
curl -s http://localhost:18084/health
```

장애 3: DB_HOST 오류

환경변수를 일부러 틀리게 실행한다.

```
docker compose down
DB_HOST=wrong-db docker compose up --build -d
sleep 5
curl -i http://localhost:18084/health
docker compose logs --tail=60 api
```

예상 해석:

관찰	의미
db_host가 wrong-db	runtime config가 잘못 들어감
name resolution 또는 connection error	service name DNS 실패
db container는 healthy일 수 있음	DB 자체 장애가 아니라 API 설정 장애

복구:

```
docker compose down
docker compose up --build -d
```

inspect로 env 확인

```
docker compose exec api env | sort | grep DB_
```

또는:

```
docker inspect $(docker compose ps -q api) --format '{{json .Config.Env}}'
```

수업에서는 exec로 상태 확인을 할 수 있지만, 운영 container를 임의로 수정하는 shell 작업은 피해야 한다. 상태 확인용 exec와 변경 작업용 exec는 다르다.

장애 리포트 미니 템플릿

항목	예시
증상	frontend /api/status 실패
영향 범위	frontend 사용자 경로, worker background check
정상 service	db healthy
의심 service	api stopped
첫 확인 명령	docker compose ps, logs frontend, logs worker
복구	docker compose start api
예방	healthcheck, alert, runbook

Evidence Note

```
# W3D1S7 Failure Drill
- injected failure:
- user visible symptom:
- first command:
- root cause candidate:
- recovery command:
- prevention note:
```

Cleanup

```
docker compose down
```

핵심 포인트

장애 분석은 많은 명령을 치는 경연이 아니다. 사용자 증상에서 시작해 dependency map을 따라가며 가장 먼저 확인할 증거를 고르는 일이다.